Agile! The Good, the Hype, and the Ugly (Bertrand Meyer)

Book Summary

Patrick Bucher

2025-12-21

Contents

This is a practical book that enables the reader to benefit from the good ideas of agile methods while staying away from the bad ones. Agile methods are a mix of horrible and great ideas. In order to spare the reader from having to try out Scrum, Extreme Programming, Lean Software, and Crystal all on his own, this book provides a description and assessment of their underlying ideas.

First, those methods are described without the usual sermons and anecdotes. Second, the underlying ideas are scrutinized and assessed, thereby separating the useful from the harmful ones. Agile methods and their texts put three difficoulties in the reader's way:

1. *Partisanship*: While it's common for their inventors to argue in favour of their inventions, proponents of agile methods ignore the rules of rational discourse and ask the reader to join their cult, which is inappropriate for engineering problems.
2. *Intimidation*: Agile texts dismiss previous approaches to software development as outdated and their users as rigid and backwards. Who wants to argue against "agile" when all of this word's opposites have negative meanings?

3. *Extremism*: Proponents of some agile methods insist on the application of a method in its entirety, which discourages a more differentiated view on the techniques that make up a specific method.

While previous books criticizing agile methods were rather timid, this book will call out the bad ideas without undue dereference.

The first chapter is a summary of agile ideas. In the second chapter, the rhetoric of agile texts is analyzed. Chapter three is a description of traditional plan-based software development—to which agile methods are the antidote. Chapters four to eight review agile ideas: principles, roles, practices, and artifacts.

Those chapters do not focus on individual methods, but on their many commonalities. Scrum, Lean, XP, and Crystal are then presented in chapter nine as combinations of those principles already discussed, each with their one single big idea emphasized.

Chapter ten describes precautions organizations should take when adopting agile methods. Chapter eleven is the final assessment; it classifies agile ideas into three categories: the good, the hype, and the ugly.

This book is not a comprehensive guide to software development, but a critique of existing approaches. Authors usually argument by gut feeling, by experience (e.g. projects saved by agile approaches and ruined by ignoring them), logical reasoning, and, ideally, empirical analysis (for which there is usually too little data available). This book relies mostly on analytical reasoning, combined with personal experience and anecdotes for illustrative purposes.

1 Overview

While agile ideas date back to the 1990s, the movement gained traction with the publication of the *Manifesto for Agile Software Development* in 2001, which was signed by many proponents of existing agile ideas. Agile software development is not a single method, but a collection of ideas grouped together to methodologies such as *Extreme Programming*, *Lean Software Development*, *Crystal*, and *Scrum*, which all select and prioritize their own subset of agile ideas. However, there are some common core characteristics to those methods:

- *Values*: general assumptions framing the agile world view
- *Principles*: organizational and technical rules based on those values
- *Roles*: responsibilities and privileges of actors involved
- *Practices*: specific activities based on said principles
- *Artifacts*: virtual and physical tools supporting those practices

1.1 Values

The fundamental assumptions of agile software development are captured in the following values:

1. *Redefined roles for developers, managers, and customers*: Some of the manager's duties are transferred to the team, such as the selection and assignment of tasks. The developers and their code are moved into the center. Customers are no longer passive recipients of a product but active participants in the development process.
2. *No "big upfront" steps*: Activities preceding the writing of code such as gathering requirements ("The customer does not know what he wants!") or creating a design ("The developers do not know what will work!") are left out because they are subject to change anyway. Instead, requirements and design emerge in a continuous process involving the customer, in which the software is iteratively refactored into his acceptance.
3. *Iterative development*: Development takes place in iterations of a fixed time, usually a few weeks, for which the functionality with the highest business value is implemented by working through a prioritized list of tasks. Functionality is added iteratively.
4. *Limited, negotiated functionality*: Only the most important features, measured by their business value, will be implemented. Unused functionality is deemed wasteful and therefore not implemented in the first place. The functionality to be added is negotiated before the start of every iteration. Since it is empirically impossible to fix both scope and deadline, usually the deadline is retained, but the scope limited accordingly.
5. *Focus on quality, understood as achieved through testing*: Quality is ensured by continuous testing rather than by upfront design decisions or by sticking to development methodologies. The project's regression test suite is a central artifact and must always pass when new functionality is added.

1.2 Principles

The following principles—not the ones from the original Manifesto, but extracted from the various texts on agile software development—turn said values into prescriptions:

- Organizational
    1. *Put the customer at the center*: Customer representatives are involved throughout the project, which should deliver the best Return on Investment to the customer.
    2. *Let the team self-organize*: The team members pick their own tasks rather than having them assigned by a manager.
    3. *Work at a sustainable pace*: Programmers work reasonable hours rather than through intense phases with long hours to meet deadlines set by a manager.
    4. *Develop minimal software*: 1) Only essential functions are built (*minimal functionality*); 2) Only what is requested is built, and extra work for future reuse and extensibility is left out (*minimal product*); 3) Only what is delivered to the customer—programs and tests—is built (*minimal artifacts*).

5. *Accept change*: Requirements are not complete at the beginning, but evolve as customers interact with intermediate releases. Change is considered normal.

- Technical

1. *Develop iteratively*: Every iteration of a fixed number of weeks produces a working release, providing the customer new functionality he can try out and give feedback on, which is then considered for a later iteration. No new functionality is demanded during an ongoing iteration; such requests are taken into consideration for an upcoming iteration instead.
2. *Treat tests as a key resource*: Quality is ensured by automated tests. No new development must start until all current tests pass. A text expresses the requirements new code must satisfy and is therefore written before the production code (*test first*).
3. *Express requirements through scenarios*: A scenario (e.g. a *use case* or a *user story*) describes an interaction of a user with the system that is to be built. Scenarios are obtained from the customer and written from a user's perspective. Unlike requirements, scenarios are not complete specifications but only examples. Scenarios do not have to be collected at the beginning of the project, but can be added and modified during development.

Thus, requirements are replaced by two artifacts: tests and scenarios.

1.3 Roles

Agile methods define various roles:

1. The *Team* is a self-organizing group of developers and other roles. Members of the team assign work items to themselves.
2. The *Product Owner* is responsible for defining the properties of the product under development. This encompasses the right to change those properties, but not while an iteration is ongoing.
3. The *Scrum Master* acts as a coach, mentor, guru, and method enforcer for the team. This role cannot be assumed by the same person that acts as the Product Owner.
4. The *Customer* is directly involved in the project or even a member of the team—rather than just being the source of requirements at the beginning and the recipient of the finished product at the end of the project.

1.4 Practices

The principles stated before are implemented using the following practices:

- Organizational

1. *Daily Meeting*: Every team member answers the following questions in a daily face-to-face meeting that takes no longer than 15 minutes: 1) "What did I do yesterday?" 2) "What do I plan to do today?" 3) "What impediments am I facing?" Those impediments are then resolved outside the daily meeting in order to keep it short.
2. *Planning Game*: Work items are estimated in a group setting, where participants are forced to come up with their own initial guess independently. Afterwards, a concensus is found iteratively in a group discussion.
3. *Continuous Integration*: Changes are integrated continuously (i.e. multiple times per day) rather than after longer development periods in order to detect incompatibilities early on.
4. *Retrospective*: The team reflects on the finished iteration in order to use the experience gained and lessons learned to improve the upcoming iteration.
5. *Shared Code Ownership*: The team as a whole, rather than individual programmers, are responsible for the entire code base. This is supposed to reduce the dependence on individuals and to avoid territorial conflicts.

- Technical

1. *Test-Driven Development*: A yet failing test case is written that describes the new functionality to be added, after which the actual functionality is implemented in order to make the new test case pass. The code can then be refactored as long as all the entire regression test suite passes.
2. *Refactoring*: The implementation is adjusted structurally in order to improve the design of the system. This step is crucial in conjunction with test-driven development to make sure that the code being written is not only working in terms of passing tests but also well-designed. Refactoring is the agile answer to big upfront design decisions of traditional methods.
3. *Pair Programming*: Code is developed by two programmers sharing a workstation in changing roles—the "driver" writing the code while explaining his thoughts, and the "navigator" trying to understand the driver's thought process in order to catch mistakes early on and to provide criticism.
4. *Simplest Solution*: Software engineering principles to improve the code's extensibility and reusability are ignored because—according to agile methods—the further direction of development cannot be known in advance. Instead, only the minimalistic product requested is built and delivered.
5. *Coding Standards*: Teams define style rules that are then applied to all the code being written.

1.5 Artifacts

Agile methods rely on a set of virtual and material artifacts:

- Virtual

1. *Use Case* and *User Story*: Both are scenarios describing an interaction of a user with a system. Use cases predate agile methods and cover an entire run through the system, whereas user stories originate from agile methods and only describe the interaction from the user's point of view.
2. *Burndown Chart*: The amount of work items due for an iteration (y-axis) is plotted against time (x-axis) to measure the *velocity*: the amount of work done per unit of time. Since the amount of work items is fixed during an iteration, the plotted line is falling as tasks are finished (and not re-opened). The plotted line can quickly be compared against a planned linear velocity denoted by a straight falling line, which informs team members of their progress (and lack thereof) at a glance.

- Material

1. *Story Card*: User stories shall be written down to small cards of the same size (3x5 inches in the US, DIN A7 in Europe), which therefore must be kept short.
2. *Story Board*: The story cards for an iteration are pinned to a story board, where the card's location indicates the underlying user story's progress, for which columns titled "todo", "in progress", "testing", "done" or the like are used.
3. *OpenRoom*: Development shall take in an open-plan setting as opposed to cubicles or closed offices, which favours interaction between team members instead of isolating them.

1.6 A First Assessment

Some of the ideas stated are genuine inventions of agile methods, while others are older and just have been re-discovered and popularized. Some of those ideas are good, others less so. Combining those qualities—new and old, good and bad—allows for clustering agile ideas into four categories:

- *not new, not good*: User stories and use cases document examples of interactions with a system. While they are useful to validate requirements, they are inadequate for replacing them. Without a proper requirements process, crucial engineering steps such as generalization and abstraction are omitted, resulting in poorly extensible and inflexible products.
- *new, not good*: While pair programming may be a useful technique in some cases, insisting on its constant use (as Extreme Programming does) ignores both different personalities of programmers and studies that fail to show its benefits. Furthermore, methods like Lean denounce useful engineering qualities such as generalization, extensibility, and automation as "waste", which might be appropriate to push back against unproductive perfection in same cases, but causes more harm then benefit when ignored altogether. Leaving out a systematic requirements process because requirements will change anyway is an inappropriate implication, because all kinds of artifacts in a software project are subject to change—but are being created nonetheless.

- *not new, good*: Iterative development with daily integration builds as well as early and frequent releases has been practiced both in commercial settings and in Open-source projects for decades. While agile methods insist on those practices, they have not invented them. The importance of embraning change is also not a new idea, but has been recognized long before. Ironically, some practices advertised by proponents of agile methods make change actually harder rather than facilitating it.
- *new, good*: Allowing a competent team to manage itself is certainly empowering. A daily meeting not only reinforces interaction between programmers, but also clarifies the next steps to be taken, and allows for early detection and resolution of impediments. Time-boxed iterations with frozen requirements during that period add stability to the development process without rendering it inflexible—change is only delayed for a couple of weeks and then handled properly. The focus on the regression test suite as a central artifact of development encourages both quality and professionalism. The focus on code as the project's main deliverable—rather than diagrams and documents—is another idea that agile methods managed to convey to the software industry at large.

2 Deconstructing Agile Texts

Literature advocating agile methods of software development often resorts to intellectually unsound methods to convince its readers.

2.1 The Plight of the Traveling Seminarist

Consider the following example from Mike Cohn's *Succeeding with Agile*, in which the author tries to convince the reader of the superiority of the spoken over the written word:

When the author asked his assistant to "book the Hyatt in Denver" by email, she responded with "the hotel is booked", by which she meant that she was unable to book a room, because the hotel was already full. However, the author interpreted her response as "your room has been booked as requested".

Cohn then argues that this misunderstanding could have been avoided had this exchange taken place by phone rather than by email. Therefore, he argues, discussions shall be favored over documents.

Ironically, this anecdote was originally presented orally in a seminar. It is way less convincing in the book's written form—where the misunderstanding becomes evident to the reader immediately. Thus, this anecdote refutes its own argument.

It is also an overstretch to conclude that discussions are better suited for software development than written requirements: This is a mere *proof by anecdote*. An anecdote, however, cannot proof a generalization; it can only proof that a generalization does *not* hold by counterexample.

Also, an opposing anecdote of a misunderstanding that could have been avoided by requiring a written specification rather than just relying on the memory of a discussion would already be a sufficient counterargument to the initial "proof" by anecdote.

Usually, such arguments just continue by invoking more anecdotes, or even worse, quotes and aphorisms, but which nothing is ever concluded.

There are many reasons to write down requirements:

- The spoken word is more ambiguous than the written one. The issue of written requirements "looking more precise than they are" is better resolved by more formal notations rather than by resorting to the even more ambiguous spoken language.
- Many discussions end with the request to "write it down", because a final decision often can only be made after reviewing a written request.
- Projects involving people from different cultures with their own English accents often suffer from misunderstandings in verbal discussions that become apparent immediately in the written form, after which the issue can usually be resolved quickly.
- Verbal discussions and their conclusions are only known to the people that attended them. A written version of the conclusions made can be shared with a wider circle.
- If decisions are made on the basis of verbal discussions, it is not certain whether or not the person making the decision is actually authorized to do so. A written document can be reviewed by the relevant stakeholders, unlike decisions being made by the last people that talked about an issue.
- People leave and join ongoing software projects. Unlike verbal discussions, written requirements survive such fluctuations.

Even though pyramids and cathedrals have been built based on mostly verbal interactions, modern engineering came a long way thanks to written requirements. Rejecting them because they are not always 100% precise is the wrong conclusion: more formal written requirements rather than fuzzy discussions are the way out.